- Step 3, **re-centering**, computing the center of mass for each cluster: we will tackle this first because here each cluster is processed independently, and computing the center of mass for a cluster only needs the points in it. We can definitely parallelize this step, with one process for each cluster, so that the execution time will be the one of the longest running thread: if the sequential version needs `n*d` sums and `k*d` divisions, where `n` is the number of points in the dataset and `d` its dimension (the cardinality of each point), assuming a uniform split to clusters (best-case scenario, of course) each process will perform `d*n/k` additions and `d` divisions. If all threads finished at the same time, and ran at the same speed as the original sequential algorithm, we would obtain a `k`-fold speed-up.
- Step 2, **classification**, is more complicated to parallelize. In theory, we would need to check all points' distances in order to assign them to the right centroid. Thinking about this more carefully though, do we really need all points? If you refer to figure 12.3 in chapter 12, it seems apparent that a point will only switch to a cluster adjacent to its current assignment, and never to one far away. Also, if a centroid `c'` moved further away from a second cluster `C` (assuming the centroid of the second cluster didn't move), it is impossible that a point in C will be assigned to c'. We would need to be very careful, though, with the assumptions we made, and so this step would be somehow more complicated to parallelize.

Even by just parallelizing step 3 of the k-means algorithm, we can obtain a nice speed-up with respect to the sequential